IT4931 / IT5384 / IT4043E · SOICT · HUST

End-to-End Lakehouse Pipeline

Lecture 14 — Capstone integration

Kafka · Spark · Iceberg · dbt · Great Expectations · Airflow

Learning outcomes

1. **Sketch** a complete lakehouse path from ingest → analytics
2. **Place** Kafka, Spark/Flink, Iceberg, dbt, GE, Airflow on one diagram
3. **Assign** bronze / silver / gold responsibilities
4. **Contrast** batch vs near-real-time paths on the **same** tables
5. **Outline** how tests + orchestration make the platform operable

Roadmap

1. **14.1** Course map · reference architecture
2. **14.2** Storage contract · medallion + Iceberg
3. **14.3** Ingest · landing to bronze
4. **14.4** Transform · Spark vs dbt
5. **14.5** Quality · GE + dbt tests
6. **14.6** Orchestration · Airflow
7. **14.7** Serving · walkthroughs · ops

14.1 Course map · reference architecture

What you already built (piecewise)

Layer	Lectures	Role today
Store	L03–L06, **L04 Iceberg**	Durable files + table contract
Query	**L07 Trino**	Multi-engine SQL on lake
Move	**L08 Kafka**	Durable event log
Patterns	**L09** Lambda · Kappa	Architecture choices
Compute	**L10–L12** Spark / Flink	Batch + stream processing
Analyze	**L13** Spark ML	Consumers of curated tables
Integrate	**L14**	One operable pipeline

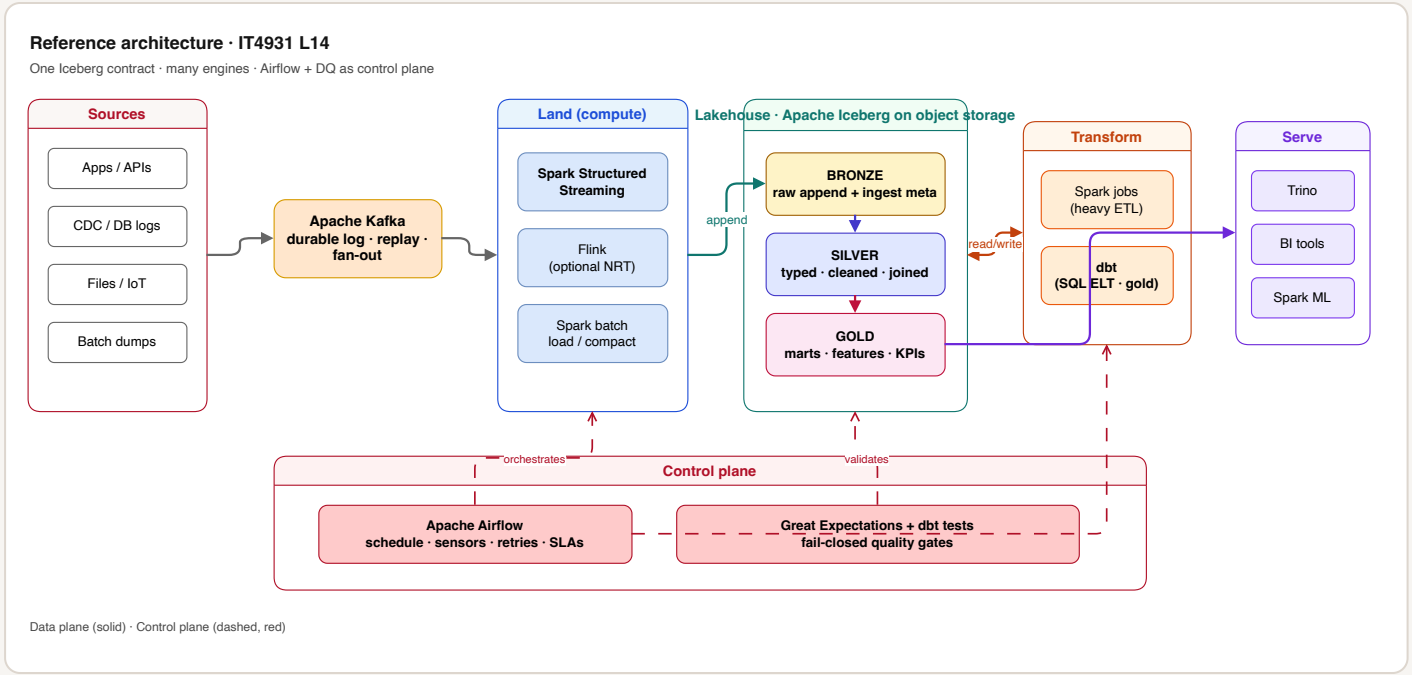
Capstone goal

Not a new storage format lecture.

Goal: show how the pieces **cooperate** in a 2026-style platform:

```
Sources → Kafka → Spark/Flink → Iceberg (bronze→gold)
                → dbt / Spark SQL transforms
                → Great Expectations + dbt tests
                → Airflow schedules & watches the DAG
                → Trino / BI / ML serve the gold layer
```

Reference architecture



One **Iceberg** contract; Airflow + DQ = control plane (dashed).

Tech stack (this lecture)

Component	Job in the pipeline
Kafka	Durable ingest · replay · fan-out
Spark	Land events · heavy transforms · ML-ready tables
Flink (L12)	Optional low-latency path into bronze/silver
Iceberg	ACID tables on object storage
dbt	SQL ELT · tests · docs · lineage on warehouse/lake
Great Expectations	Data-quality suites beyond SQL asserts
Airflow	Schedule, sensors, retries, lineage of *jobs*

Lambda / Kappa → this stack

Pattern (L09)	How it shows up here
Lambda	Batch Spark rebuilds gold; stream path updates speed views
Kappa	Kafka log + one stream processor (Spark SS / Flink) + replay
Lakehouse	Both paths write **Iceberg**; serving reads one table format

L14 picks a **pragmatic hybrid**: stream into bronze, batch/dbt for gold, optional stream features.

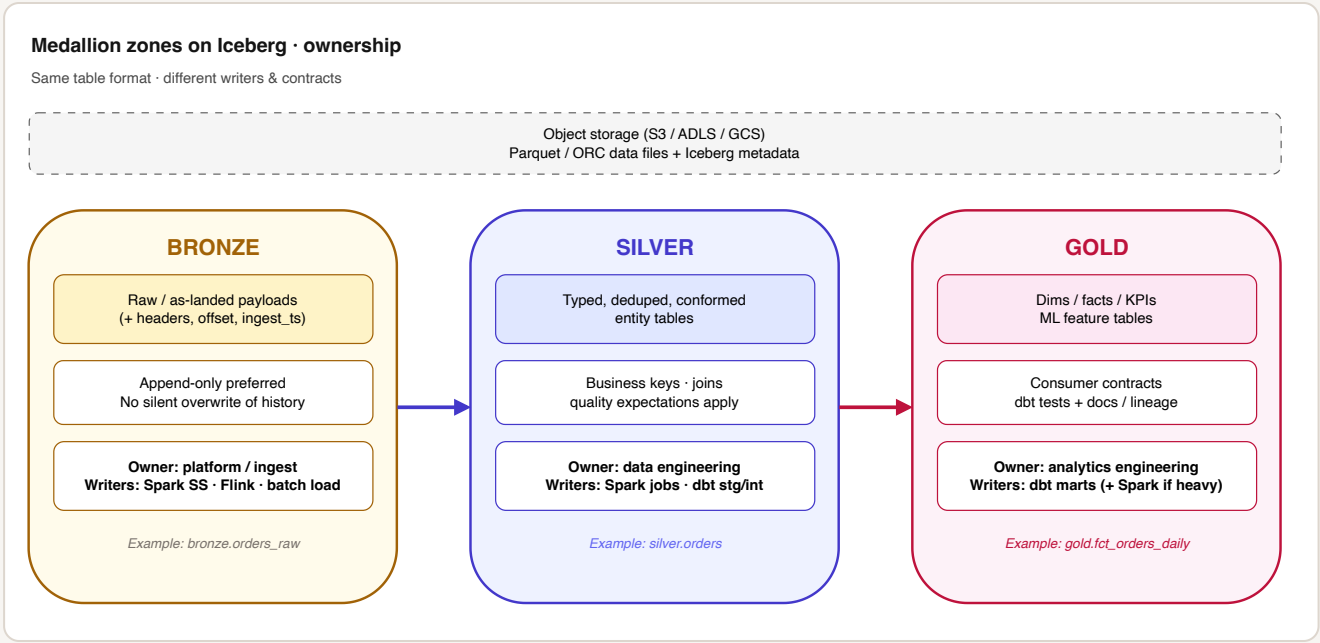
14.2 Storage contract · medallion + Iceberg

Recall · lakehouse (L04)

- Object storage holds **files**; **Iceberg** is the metadata + transaction contract
- Engines (Spark, Trino, Flink, dbt adapters) share tables
- Time travel / snapshots enable safe reprocessing

Today we only need: **where data lands** and **who is allowed to mutate which layer**.

Medallion on Iceberg



Airflow DAGs and repo folders should mirror these zones.

Why Iceberg as the hub

Need	Iceberg helps
Multi-engine	Spark writes; Trino / dbt read
Concurrent writers	Snapshot isolation
Reprocess	Time travel + selective rewrite
Schema change	Evolution without path rewrites

Rule of thumb: put the contract in Iceberg; keep business SQL in dbt/Spark — not in ad-hoc notebooks alone.

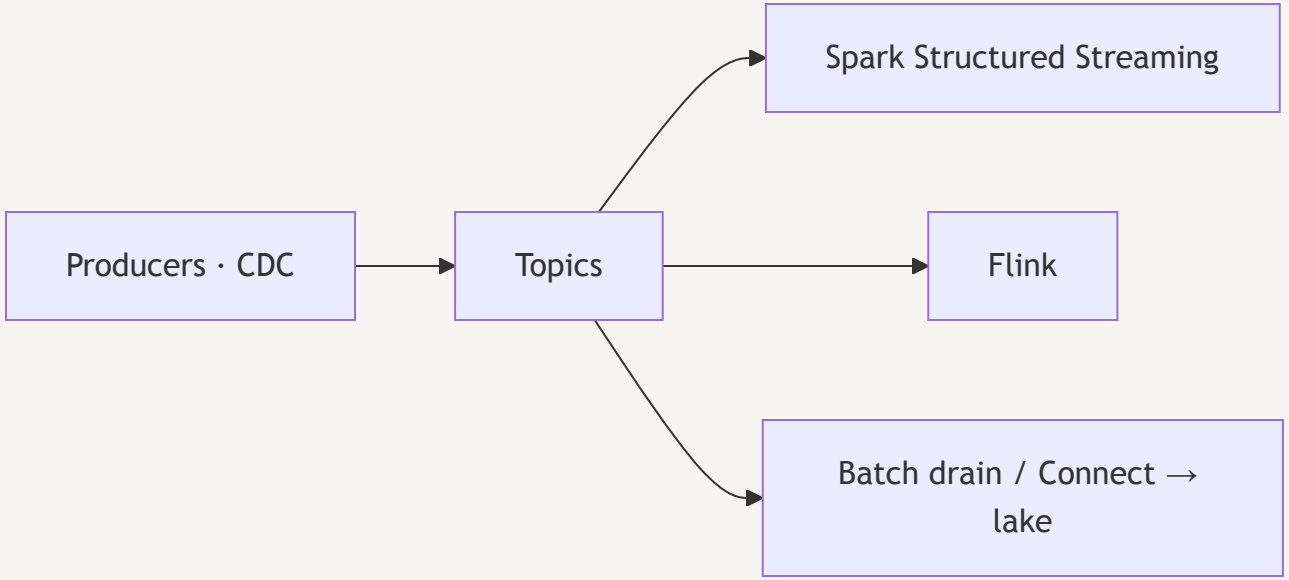
Naming & ownership

Zone	Example	Owner mindset
`bronze.*`	`bronze.orders_raw`	Platform / ingest team
`silver.*`	`silver.orders`	Data engineering
`gold.*`	`gold.fct_orders_daily`	Analytics engineering (dbt)

Airflow DAGs and repo folders should **mirror** these zones.

14.3 Ingest · landing to bronze

Kafka as the front door (L08)



Why Kafka first	Detail
Buffer	Decouple producers from lake writers
Replay	Rebuild bronze from offsets / time
Fan-out	Same events → search, stream apps, lake

Landing job · conceptual Spark SS

```
# Conceptual — land Kafka → Iceberg bronze
df = (spark.readStream
      .format("kafka")
      .option("subscribe", "orders")
      .load())

parsed = parse_and_enrich(df)   # schema, ingest_ts, offset

(parsed.writeStream
   .format("iceberg")
   .outputMode("append")
   .option("checkpointLocation", "s3://…/_checkpoints/orders")
   .toTable("bronze.orders_raw"))
```

Checkpoint + Iceberg commit ⇒ recoverable landing (ties to L11 EOS ideas).

When Flink lands instead

Prefer Spark SS	Prefer Flink (L12)
Team already on Spark	Sub-second / complex keyed state
Micro-batch OK	Event-time timers / CEP
Same runtime as lake ETL	Dedicated stream platform

Both should still target **Iceberg bronze** (or files + compaction job) — don't invent a second "source of truth".

Bronze hygiene

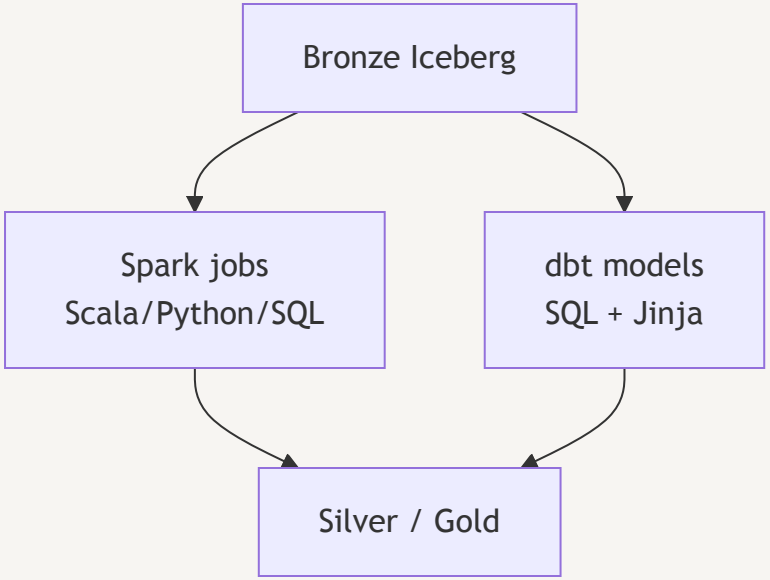
Keep bronze **faithful** to the source:

- Preserve raw payload (+ headers / partition / offset)
- Add **ingest timestamp** and job id
- Prefer **append-only**; correct via new snapshots, not silent overwrite
- Compact / expire old snapshots on a schedule (Airflow)

Silver is where you **interpret**; bronze is where you **prove** what arrived.

14.4 Transform · Spark vs dbt

Two transform styles



Style	Strength
Spark	Large joins, custom UDFs, ML feature eng, streaming
dbt	Versioned SQL, tests, docs, lineage, analytics marts

Many platforms use **both**: Spark for heavy silver; dbt for gold marts.

What is dbt? (in this pipeline)

Piece	Meaning
dbt	Write `SELECT` (+ Jinja) → tables/views in the warehouse/lake
dbt Core	CLI in CI / Airflow / containers
Adapters	Snowflake, BQ, Spark, Trino, Databricks, …
Not dbt	Ingestion, Kafka, stream CEP

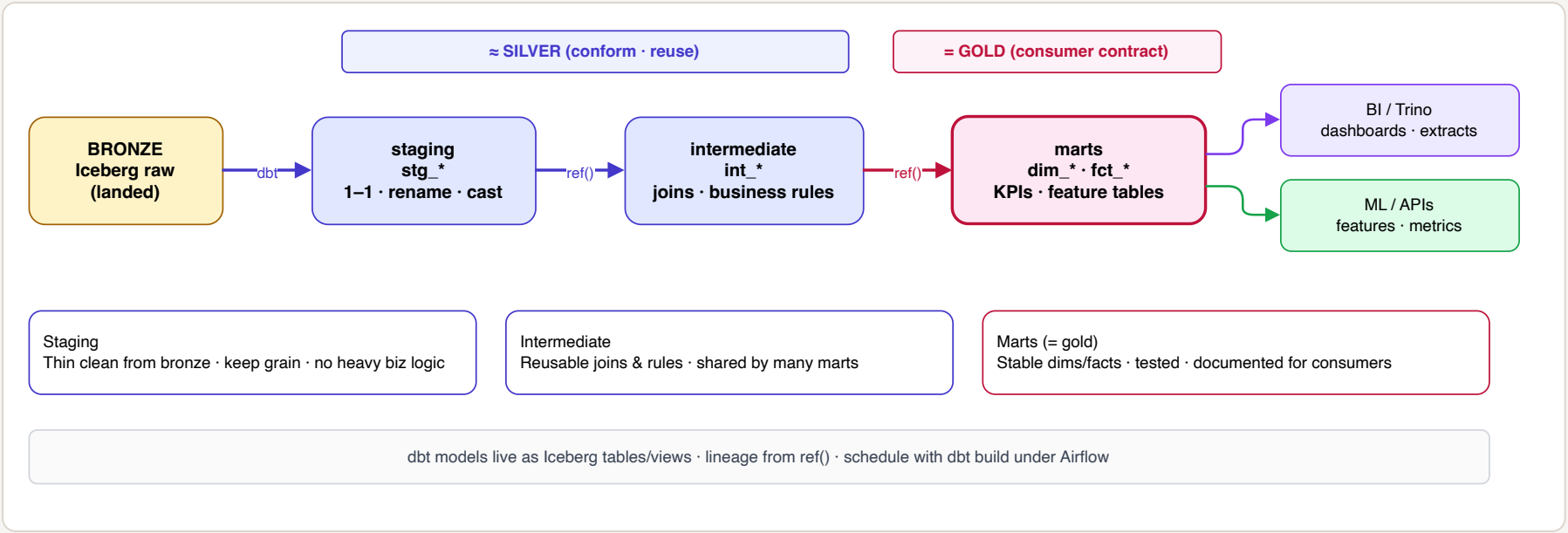
dbt is the **T in ELT** on data **already** in Iceberg/warehouse.

dbt project shape

```
models/
  staging/        stg_*     ← bronze → thin clean
  intermediate/   int_*     ← joins / business rules
  marts/          dim_*/fct_*
macros/
tests/            (or tests in .yml)
dbt_project.yml
```

Commands you will schedule: `dbt run` · `dbt test` · `dbt build` · `dbt docs generate`

Staging → intermediate → marts



dbt folder layout ≈ medallion: `stg_*` / `int_*` ≈ silver · `dim_*` / `fct_*` = gold. Serve consumers from marts — not from bronze.

Example · gold fact (sketch)

```sql
-- models/marts/fct_orders_daily.sql
select
  order_date,
  customer_id,
  count(*) as order_cnt,
  sum(amount) as gmv
from {{ ref('int_orders_enriched') }}
group by 1, 2
```

`ref()` builds the **lineage graph** Airflow/docs will show.

Materialize as **Iceberg** table when the adapter supports it.

Spark still required when…

Situation	Prefer
Multi-TB shuffle-heavy joins	Spark job
Streaming window features	Spark SS / Flink
Non-SQL transforms / graph / custom ML prep	Spark (→ L13)
Analyst-owned marts + tests	**dbt**

Don’t force everything through dbt — and don’t leave gold only in notebooks.

14.5 Quality · Great Expectations + dbt tests

Why quality is a first-class stage

Broken gold silently trains bad models and wrong dashboards.

Layer	Typical checks
Bronze	Volume, schema drift, null spike on required fields
Silver	Uniqueness, referential integrity, ranges
Gold	KPI sanity, freshness, row-count vs yesterday

Quality gates belong **in the DAG**, not in Slack after the fact.

dbt tests (declarative)

```yaml
# models/marts/schema.yml
models:
  - name: fct_orders_daily
    columns:
      - name: order_date
        tests: [not_null]
      - name: customer_id
        tests: [not_null, relationships]
      - name: gmv
        tests:
          - dbt_utils.accepted_range:
              min_value: 0
```

`dbt test` / `dbt build` fails the Airflow task → no silent publish.

Great Expectations (GE)

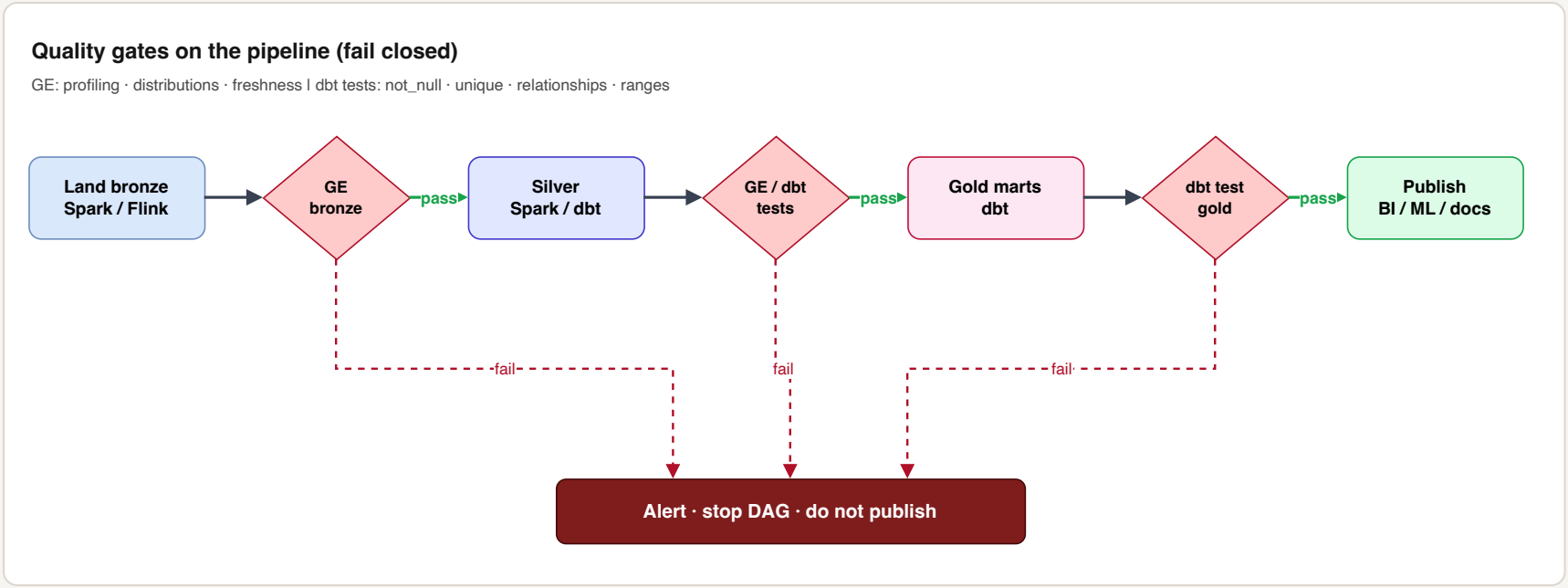
Idea	Detail
Expectation suite	Reusable assertions on a batch of data
Checkpoint	Run suite → store results → optionally fail pipeline
Data Docs	Human-readable validation report
Fit	Profiling, statistical checks, multi-batch freshness

Use GE when checks outgrow simple SQL tests — or for bronze profiling before dbt.

dbt tests vs GE

	dbt tests	Great Expectations
Authoring	YAML next to models	Suites / Python / GE Cloud
Best for	Mart contracts · FK · nulls	Profiling · distribution · cross-batch
Runtime	Inside `dbt build`	Separate task (often Spark/Pandas)
Together	dbt owns gold contracts; GE owns landing & deep DQ	

Quality placement on the DAG



14.6 Orchestration · Airflow

Why Airflow here

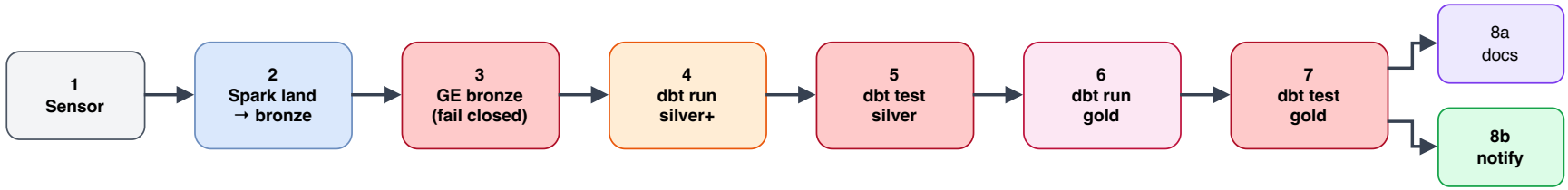
Without orchestrator	With Airflow
Cron soup · manual notebooks	One DAG = one business pipeline
Unclear dependencies	Explicit task graph + sensors
Weak retries / alerting	Retries, SLAs, callbacks
Hard audit	Run history · logs · lineage of jobs

Airflow does **not** replace Spark/dbt — it **schedules and watches** them.

Example DAG · daily gold

Airflow DAG · daily gold (conceptual)

Thin orchestrator · thick Spark / dbt / GE jobs · any red failure stops publish



Grey=sensor · Blue=compute · Orange=dbt transform · Red=quality · Pink=gold · Purple=docs · Green=publish

Sensor example: Kafka lag OK / partition ready

Airflow task sketch

```
# Conceptual DAG fragment
land = SparkSubmitOperator(task_id="land_orders", …)
ge_bronze = BashOperator(task_id="ge_bronze",
                         bash_command="great_expectations checkpoint run …")
dbt_build = BashOperator(task_id="dbt_build",
                         bash_command="dbt build --select tag:orders")

land >> ge_bronze >> dbt_build
```

Operators wrap **existing** jobs; business logic stays in Spark/dbt repos.

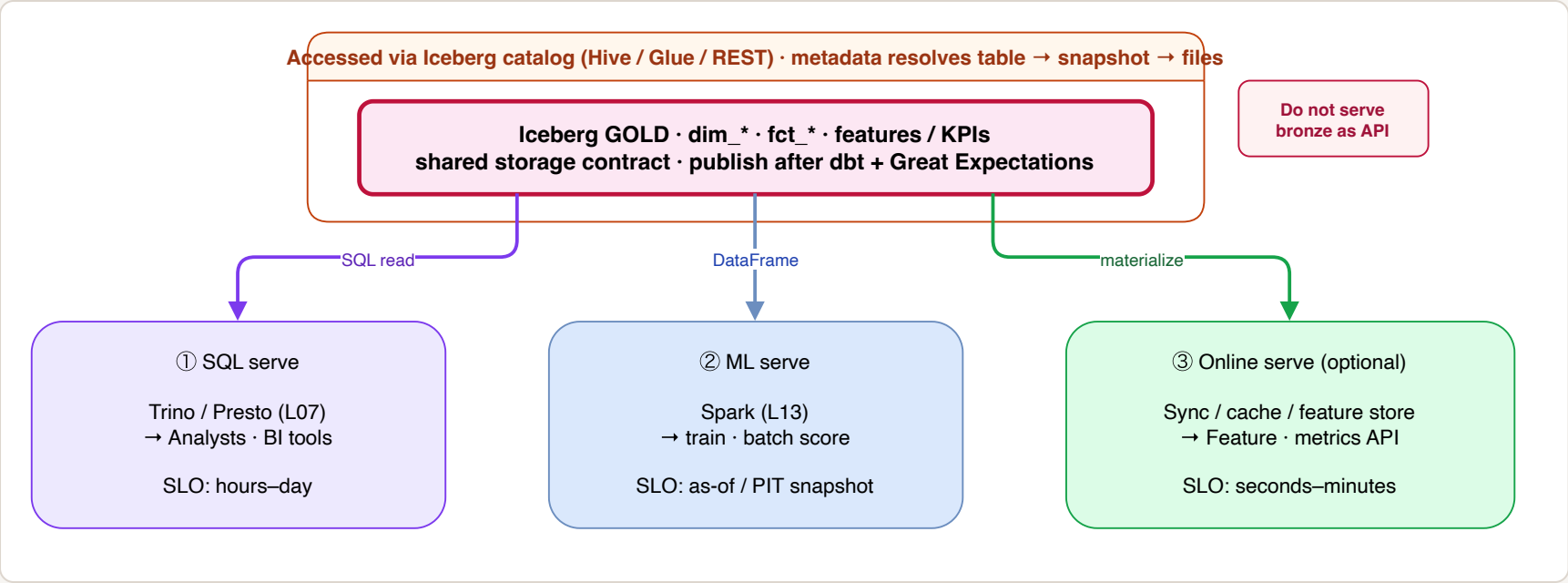
Streaming jobs vs Airflow

Workload	Orchestration style
Long-running Spark SS / Flink	Deploy once; Airflow **monitors** health / restarts
Batch land / compact / dbt	Airflow **triggers** on schedule or sensor
Backfill	Airflow params + Iceberg time travel / Kafka replay

Don’t stop a healthy stream every hour “because Airflow ran” — separate **control plane** from **data plane**.

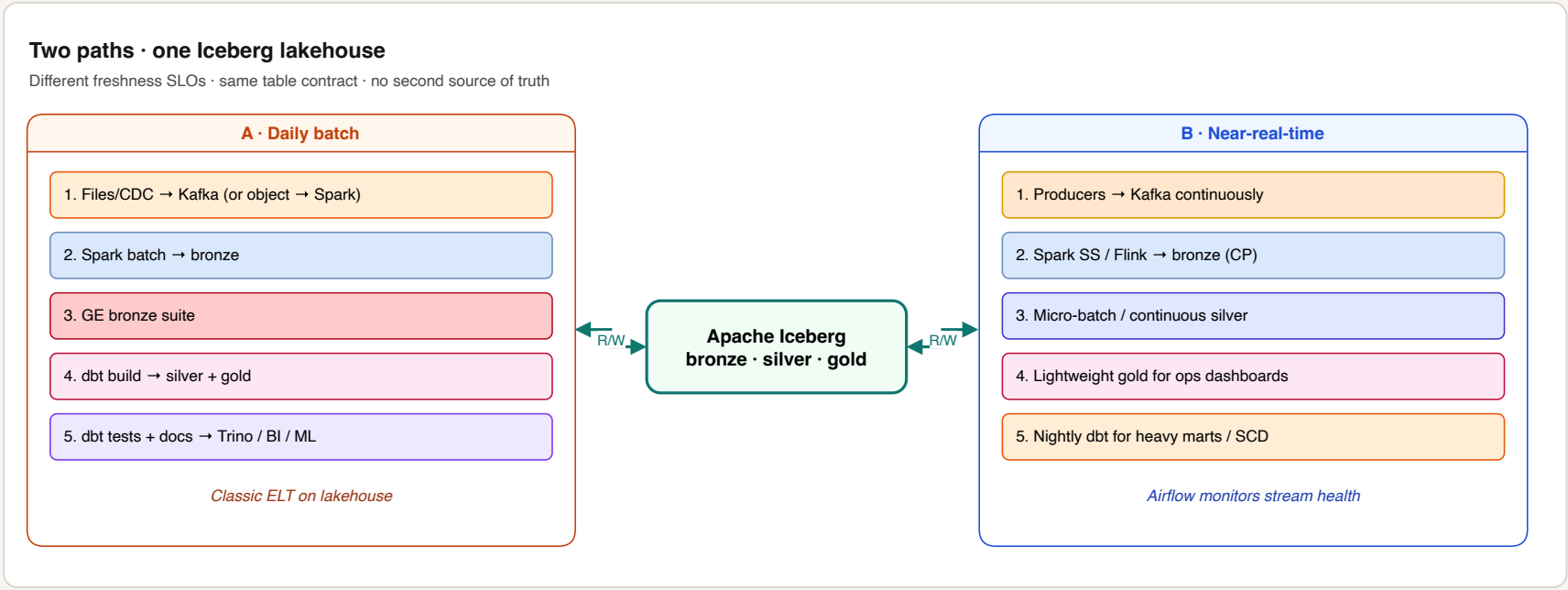
14.7 Serving · walkthroughs · ops

Serving the gold layer



One GOLD contract · three serve paths: SQL (Trino→BI) · ML (Spark as-of) · Online (materialize/cache when lake scan is too slow). Catalog is metadata only — never serve bronze as an API.

Batch vs near-real-time



Walkthrough A · daily batch path

1. Files / CDC dump land in Kafka (or object → Spark batch)
2. Spark batch appends **bronze** Iceberg
3. GE bronze suite passes
4. `dbt build` refreshes silver + gold
5. dbt tests + docs
6. Trino / BI see new snapshot; ML training job may run

Closest to classic **ELT on lakehouse**.

Walkthrough B · near-real-time path

1. Producers → Kafka continuously
2. Spark SS / Flink append **bronze** (checkpointed)
3. Micro-batch or continuous silver job
4. Lightweight gold (or stream aggregates) for ops dashboards
5. Nightly dbt rebuild for heavy marts / SCD

Same **Iceberg tables**; different **freshness SLOs** per consumer.

End-to-end picture (ops view)

Concern	Practice
Freshness	SLAs in Airflow · lag metrics on Kafka
Correctness	GE + dbt tests · fail closed
Reprocess	Kafka replay · Iceberg time travel · dbt `--full-refresh`
Cost	Compact Iceberg · prune snapshots · right-size Spark
Governance	Zone ownership · docs · access on gold

Anti-patterns

Anti-pattern	Prefer
Notebooks as production gold	dbt / reviewed Spark jobs
Overwrite bronze “to fix”	Append + corrective silver
Airflow embeds giant SQL	Call dbt/Spark; keep DAG thin
One mega-DAG for the company	Domain DAGs + shared datasets
Skip tests “to go faster”	Smaller suites, still gated

Project ideas (course)

- Mini lakehouse: Kafka topic → Spark land → Iceberg → dbt marts
- Add one GE suite + Airflow DAG with fail-on-error
- Compare batch-only vs streaming bronze on the same mart
- Trino query gold; optional Spark ML consumer (L13)

Demonstrate **integration**, not every advanced feature of each tool.

Course wrap · storage & processing

1. **Store** — HDFS, NoSQL, object + **lakehouse / Iceberg**
2. **Move** — **Kafka**
3. **Process** — Spark batch/stream · Flink · Lambda/Kappa
4. **Query / analyze** — Trino · Spark ML
5. **Integrate** — **dbt + GE + Airflow** on one pipeline

You can now **draw and defend** a complete platform design.

Takeaways

1. Capstone = **one architecture**, many engines sharing **Iceberg**
2. Kafka buffers; Spark/Flink land; dbt owns SQL gold
3. **GE + dbt tests** gate publish; Airflow runs the graph
4. Batch and streaming are **paths**, not separate lakes
5. Thin orchestrator · thick versioned transforms

End of lecture series

IT4931 / IT5384 / IT4043E
Big Data Storage and Processing

From HDFS & Kafka to a full **lakehouse pipeline**.